

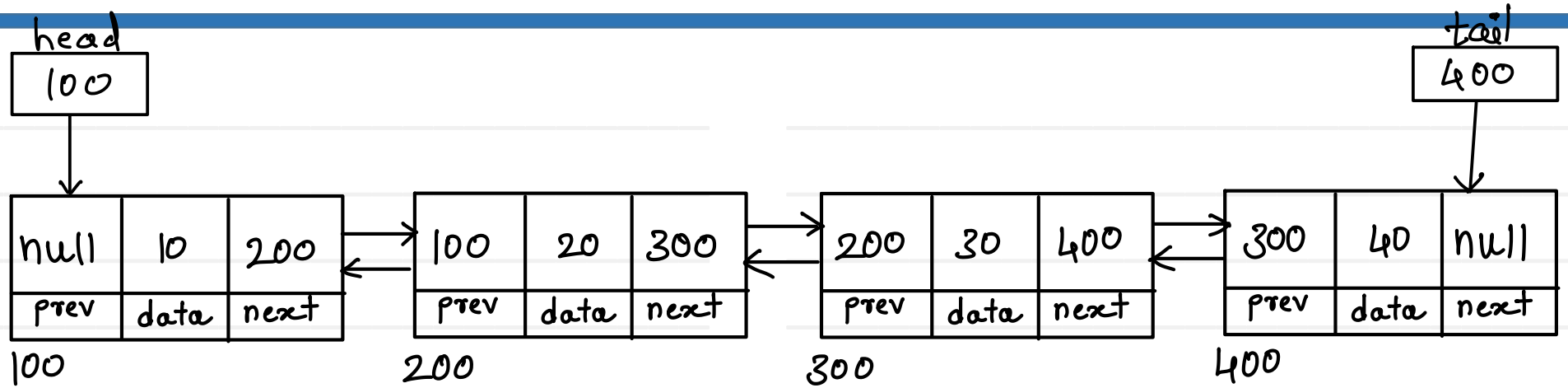
Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

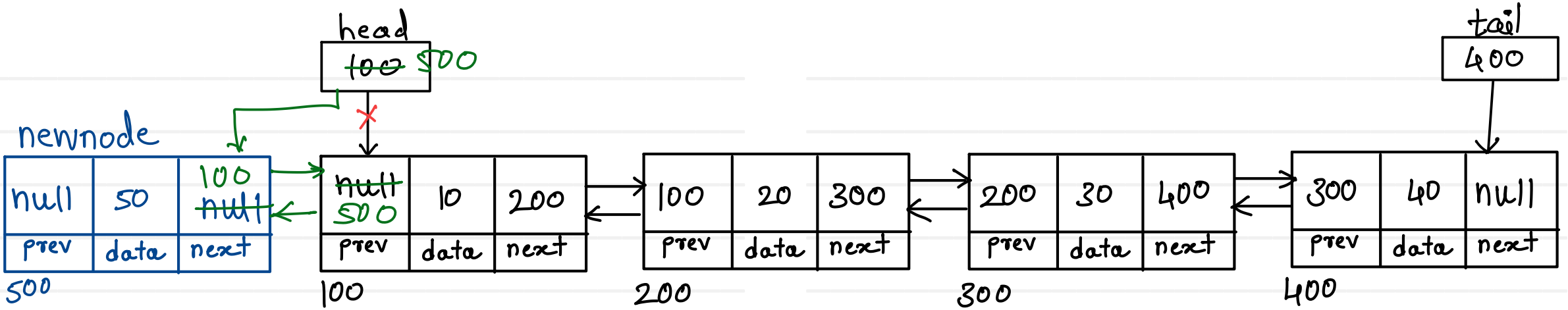
Doubly Linear Linked List - Display

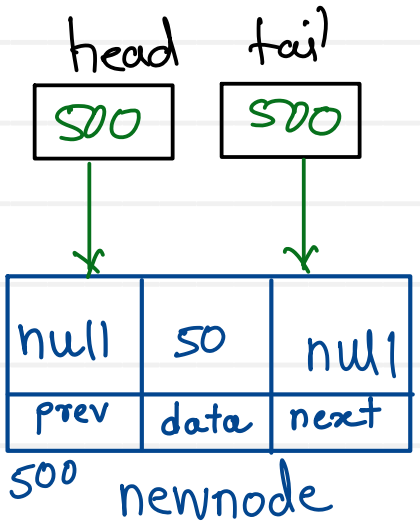


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

Doubly Linear Linked List - Add first

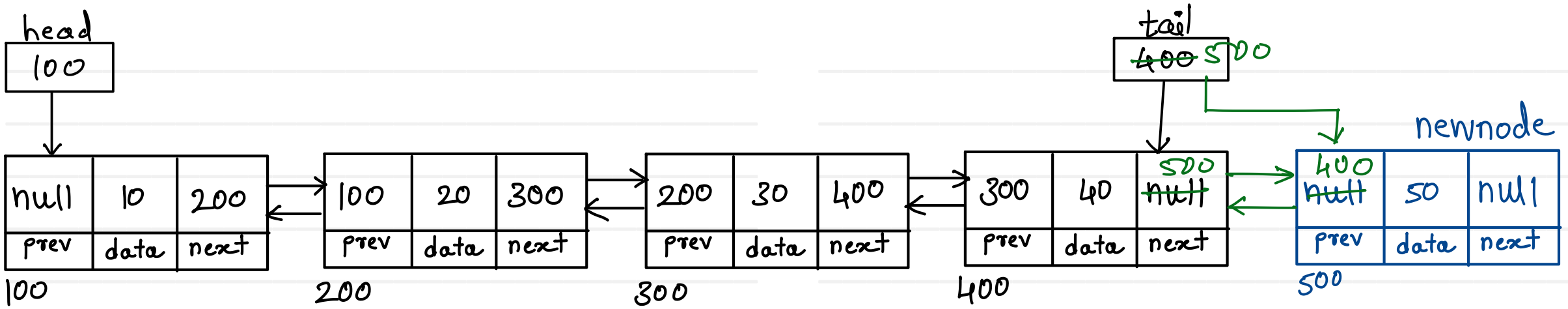


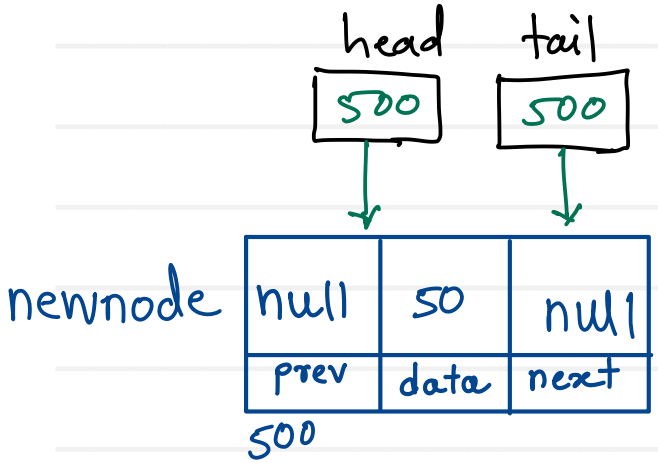


1. create a newnode
2. if list is empty
   head = tail = newnode
3. if (list is not empty
   a. add first into next of newnode
   b. add newnode into prev of first node
   c. move head on newnode

$T(n) = O(1)$

Doubly Linear Linked List - Add last

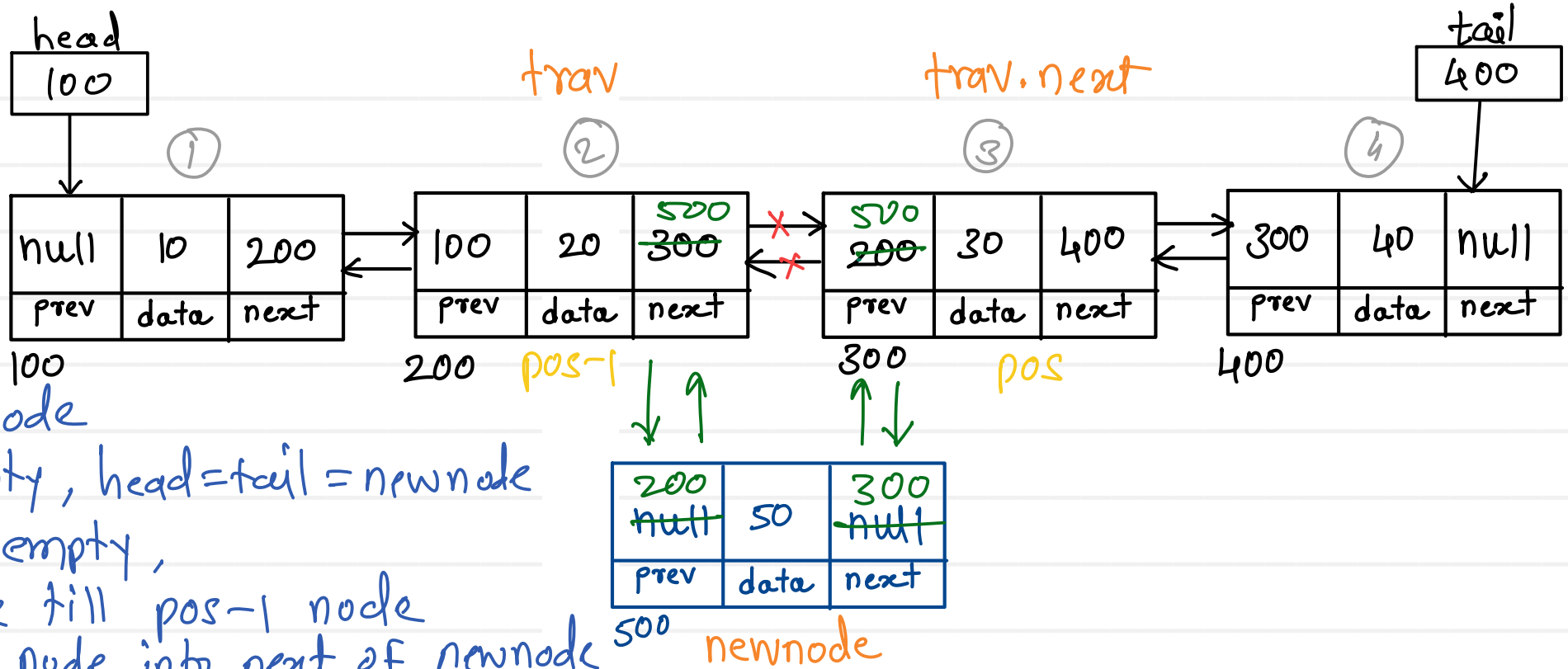




1. create a newnode
2. if empty, head = tail = newnode
3. if not empty,
   a. add last node into prev of newnode
   b. add newnode into next of last node
   c. move tail on newnode

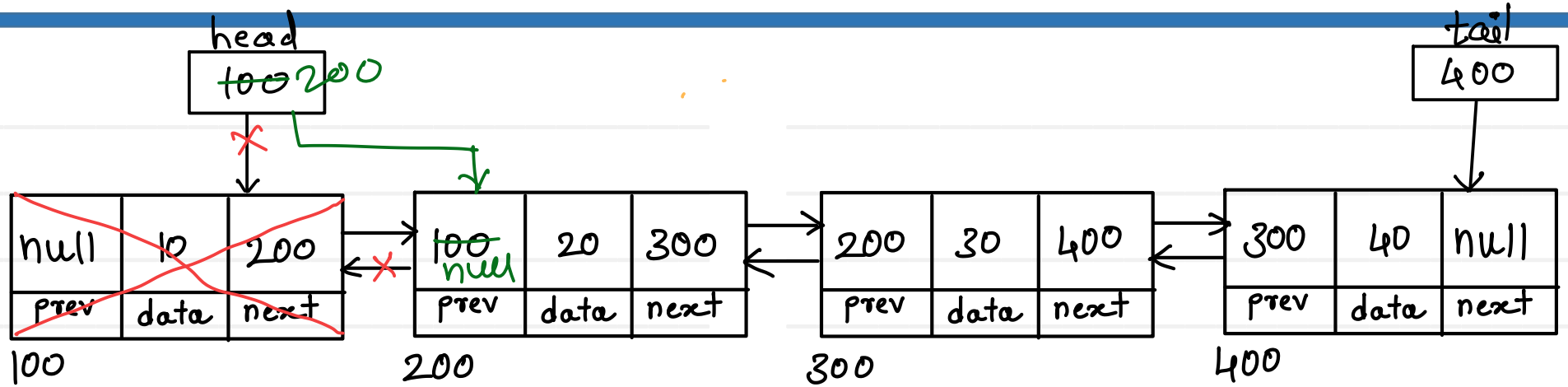
Doubly Linear Linked List - Add position

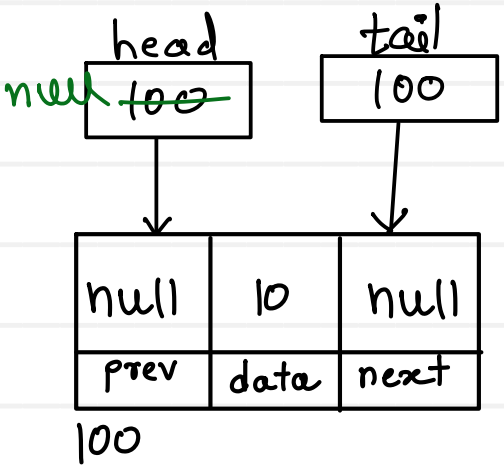
pos = 3



1. Create a newnode
2. if list is empty, head=tail=newnode
3. if list is not empty,
   a. traverse till pos-1 node
   b. add pos node into next of newnode
   c. add pos-1 node into prev of newnode
   d. add newnode into prev of pos node
   e. add newnode into next of pos-1 node

Doubly Linear Linked List - Delete first





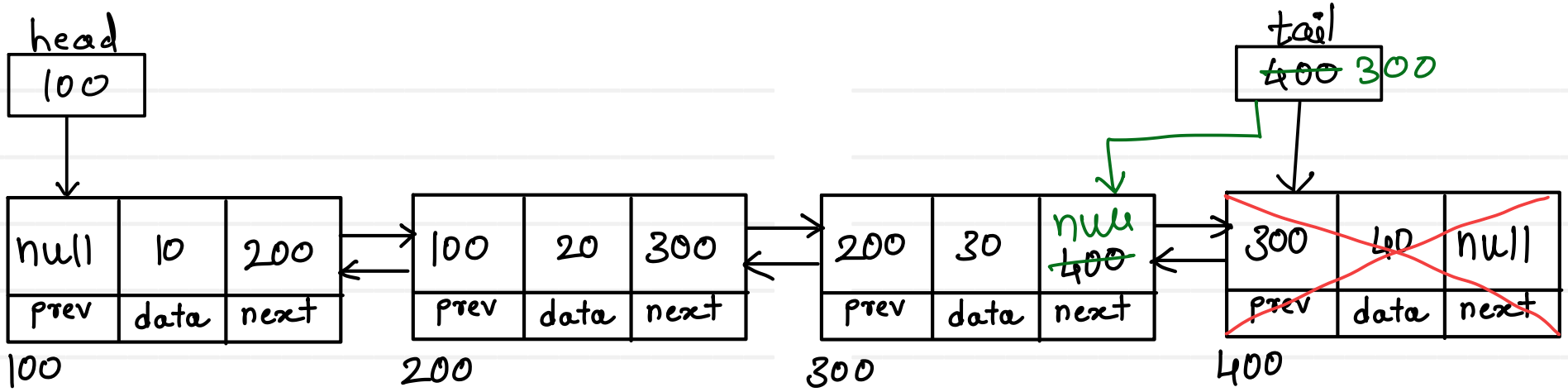

```
head = head.next;
head.prev = null;
```

null pointer exception

1. if empty, return
2. if single node, head = tail = null
3. if multiple nodes,
   a. move head on second node
   b. make prev of second node null

$T(n) = O(1)$

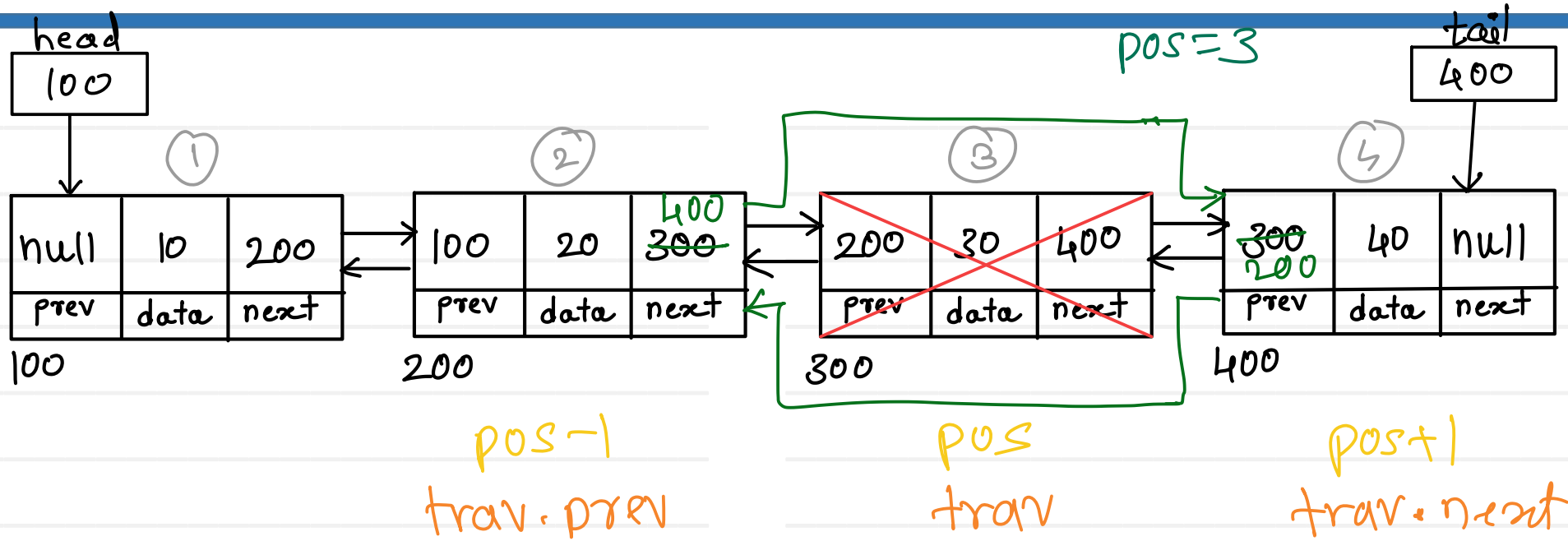
Doubly Linear Linked List - Delete last



1. if list is empty, return
2. if list has single node, delete it
3. if list has multiple nodes
   a. move tail on second last node
   b. add null into next of second last node

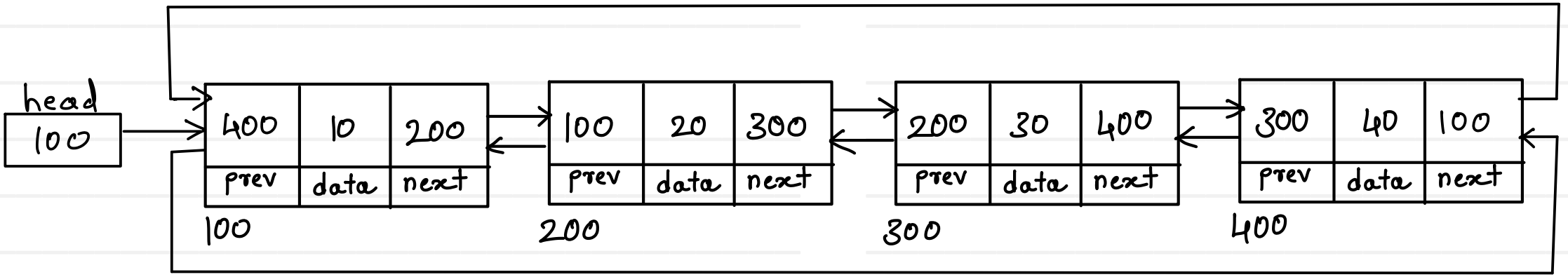
$T(n) = O(1)$

Doubly Linear Linked List - Delete Position



3. if multiple nodes,
   a. traverse till pos node
   b. add pos+1 node into next of pos-1 node
   c. add pos-1 node into prev of pos+1 node

Doubly Circular Linked List - Display

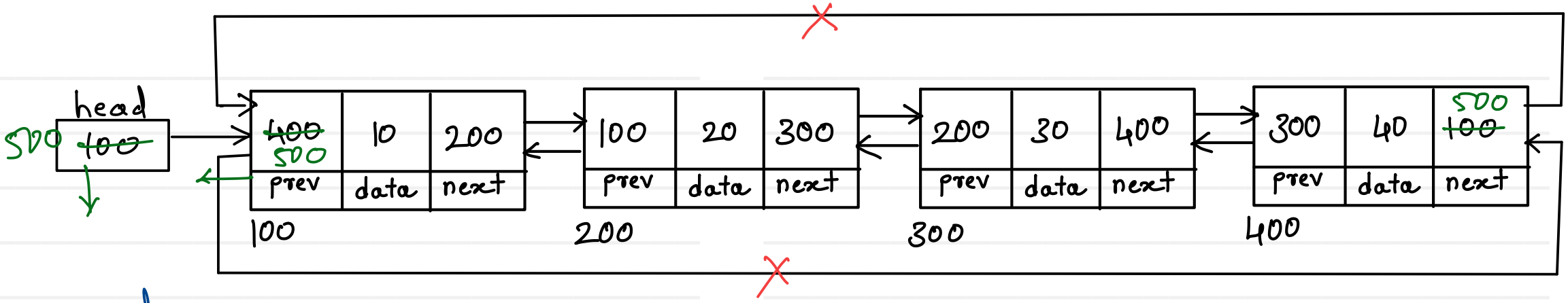


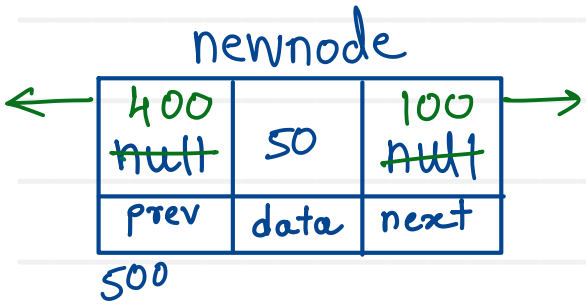
1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

$$T(n) = O(n)$$

Doubly Circular Linked List - Add first





if list is not empty,

a. newnode.next = first node

b. newnode.prev = last node

c. (last node).next = newnode

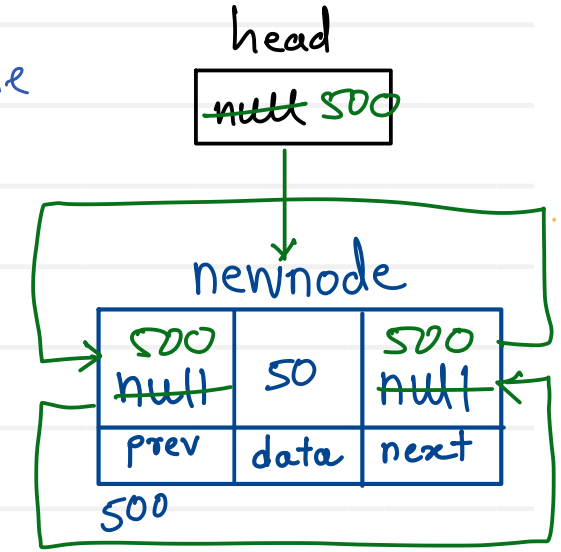
d. (first node).prev = newnode

e. head = newnode

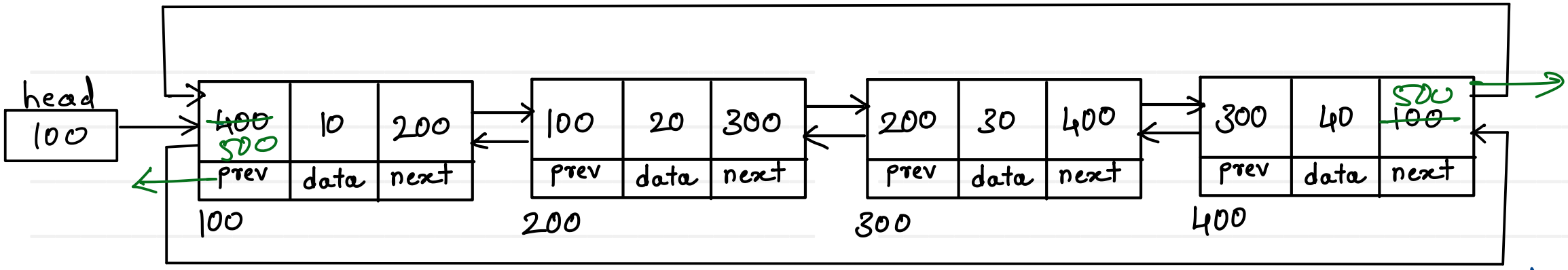
if list empty,

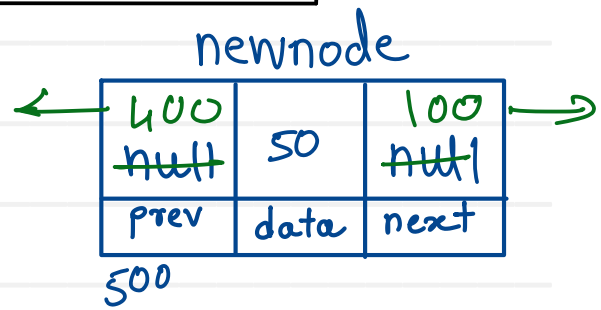
a. add newnode into head

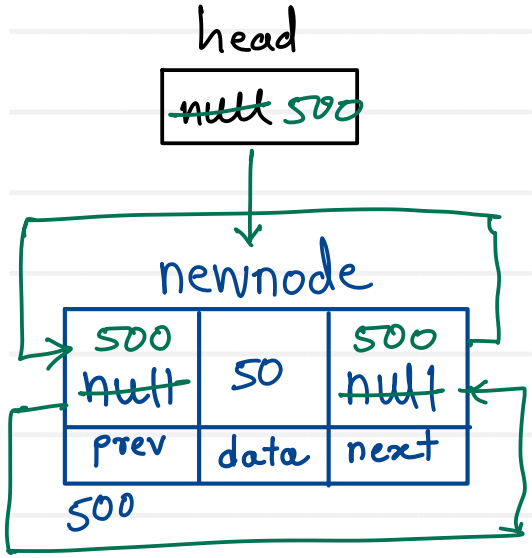
b. make list circular



Doubly Circular Linked List - Add last



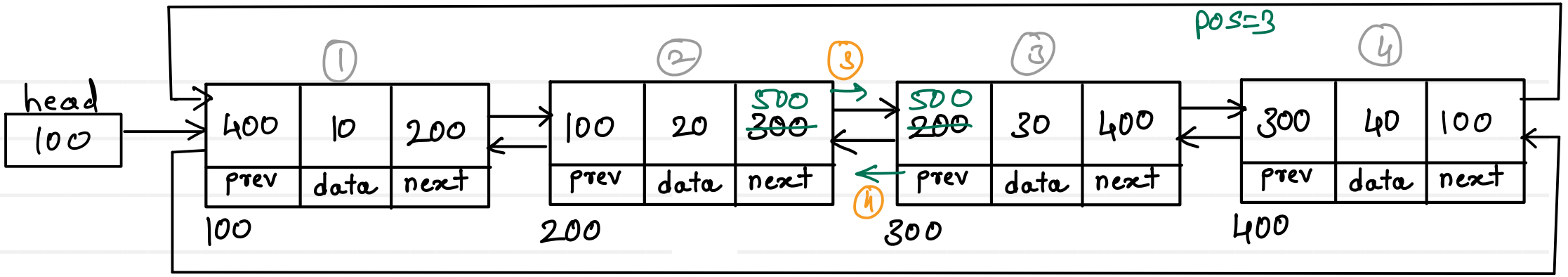




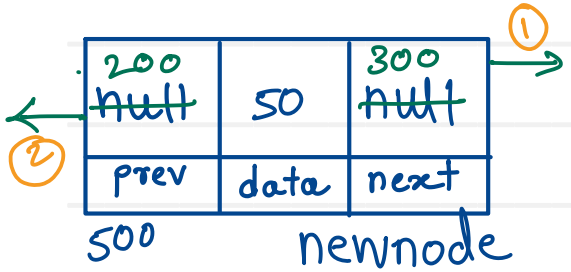
if list is not empty,
a. newnode.next = first node
b. newnode.prev = last node
c. (last node).next = newnode
d. (first node).prev = newnode

if list empty,
a. add newnode into head
b. make list circular

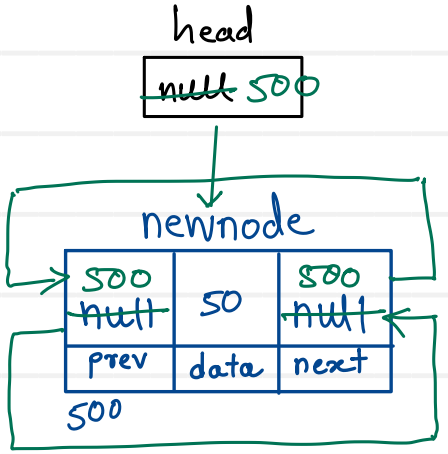
Doubly Circular Linked List - Add position



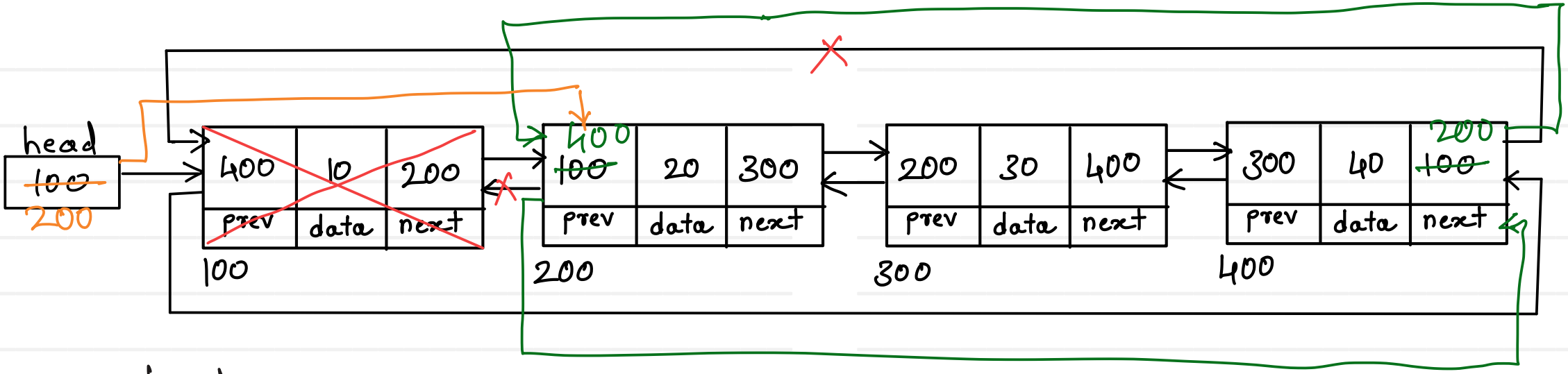
1. create node
2. if list is empty
   a. add newnode into head
   b. make list circular
3. if list is not empty.
   a. traverse till pos-1 node
   b. add pos node into next of newnode
   c. add pos-1 node into prev of newnode
   d. add newnode into next of pos-1 node
   e. add newnode into prev of pos node

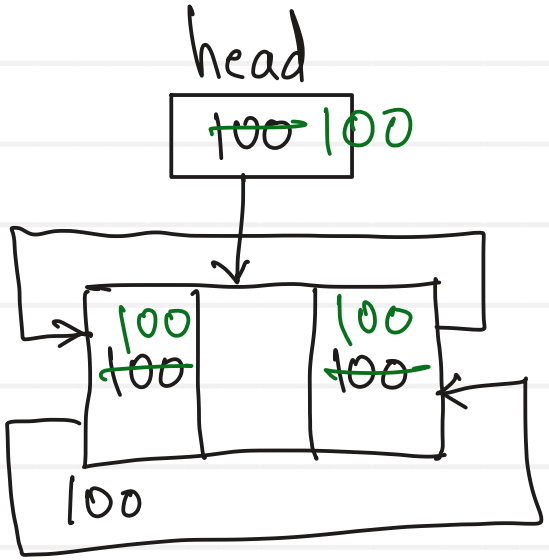


$T(n) = O(1)$



Doubly Circular Linked List - Delete first

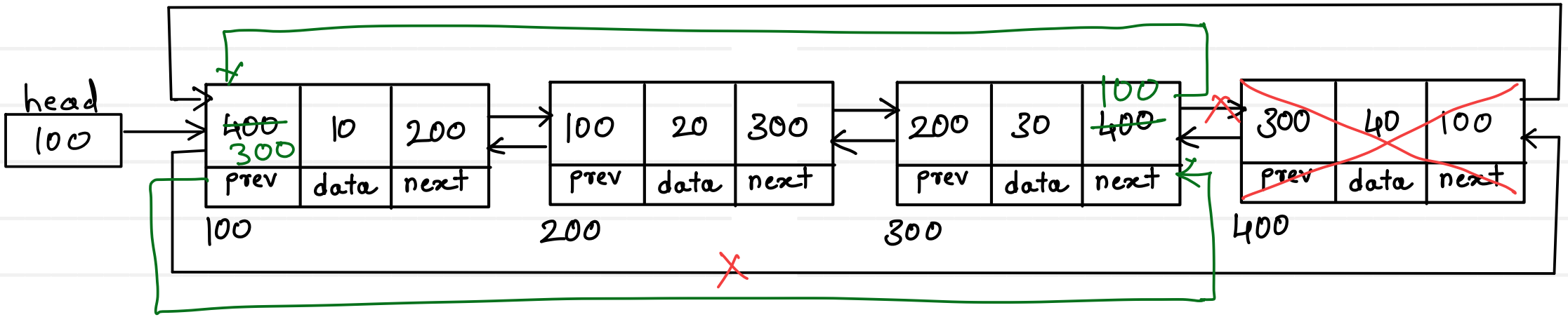





```
head.next.prev = head.prev;
head.prev.next = head.next;
head = head.next;
```

1. if empty, return
2. if single node, delete it
3. if multiple nodes,
   a. add last node into prev of second node
   b. add second node into next of last node
   c. move head second node

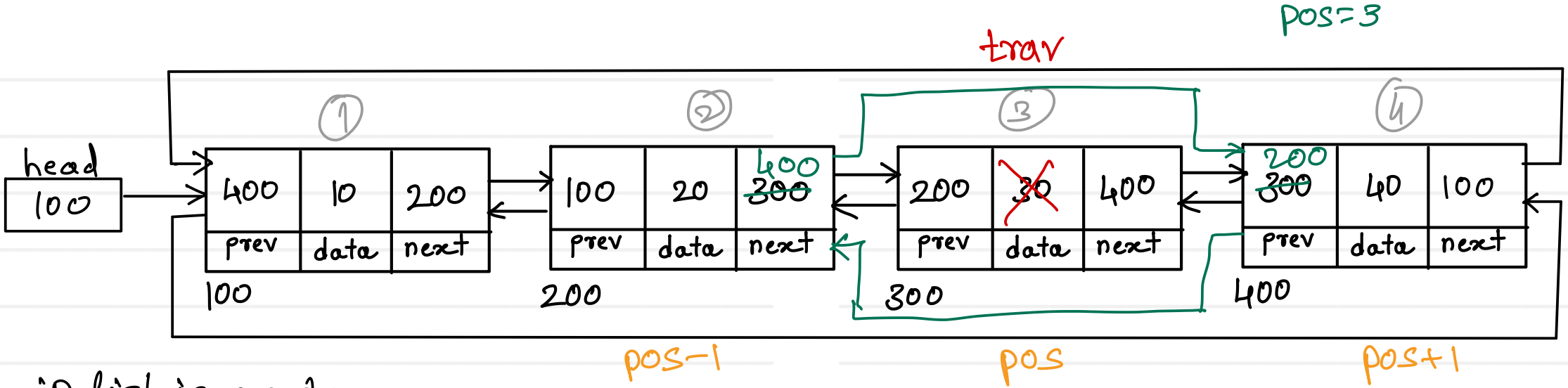
Doubly Circular Linked List - Delete last



if list has multiple nodes,

1. add first node into next of second last node
2. add second last node into prev of first node

Doubly Circular Linked List - Delete position



1. if list is empty
   return;
2. if list has single node
   head = tail = null;
3. if list has multiple nodes
   a. traverse till pos node
   b. add pos+1 node into next of pos-1 node
   c. add pos-1 node into prev of pos+1 node

trav -> pos
trav.prev -> pos-1
trav.next -> pos+1

$T(n) = O(n)$

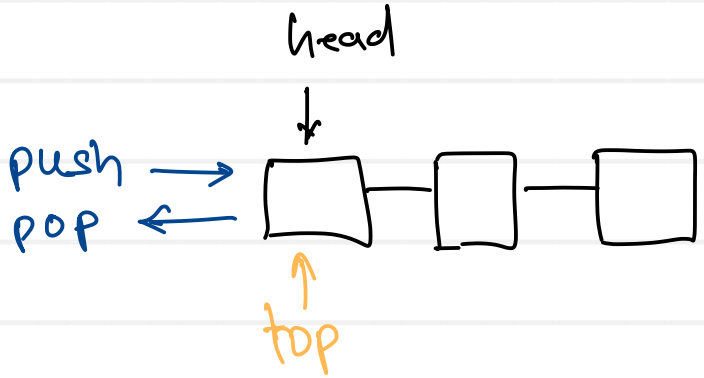
Stack and Queue using linked list

Stack

- single end : top
- LIFO

1. Add/Delete first
2. Add/Delete last

Time complexity of operations = O(1)

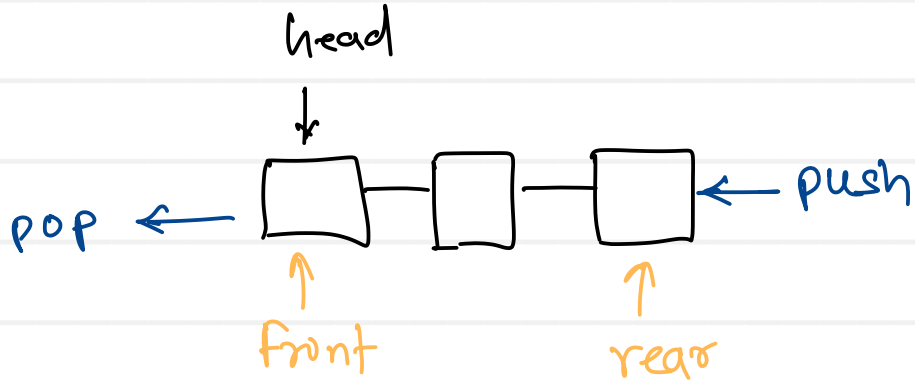


Queue

- two ends : front, rear
- FIFO

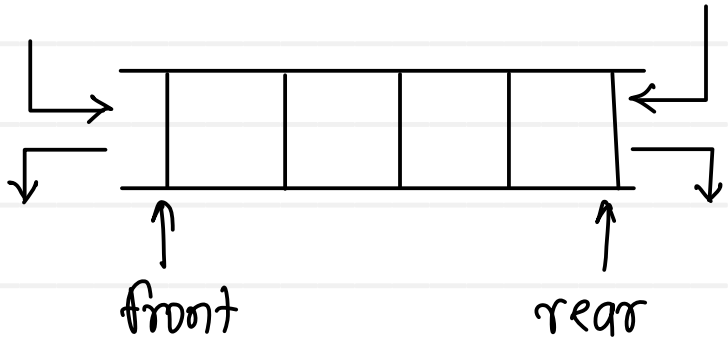
1. Add first, Delete last
2. Add last, Delete first

Time complexity of operations = O(1)



Double Ended Queue - Deque

- data can be inserted or removed in a queue from both ends



- Deque do not follow "FIFO" principle

- to implement double ended queue, we can use circular array or linked list

Operations :

1. pushFront()
2. popFront()
3. peekFront()
4. pushRear()
5. popRear()
6. peekRear()

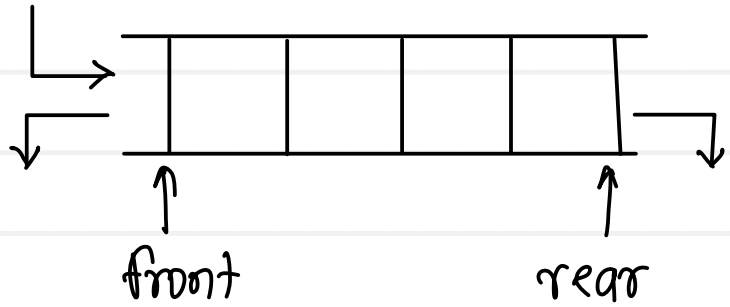
7. isEmpty()
8. isFull()

Types of Deque

Input Restricted Deque

remove from both ends

insert from only one end

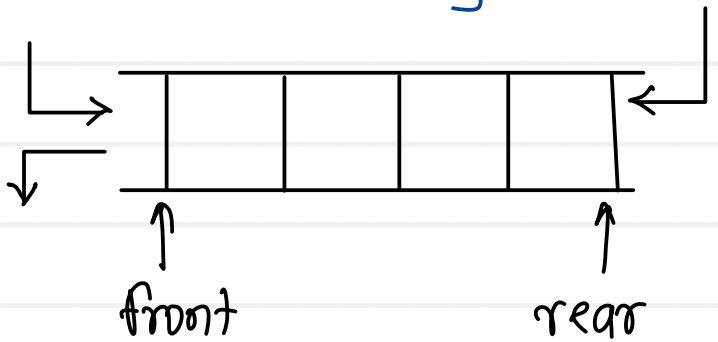


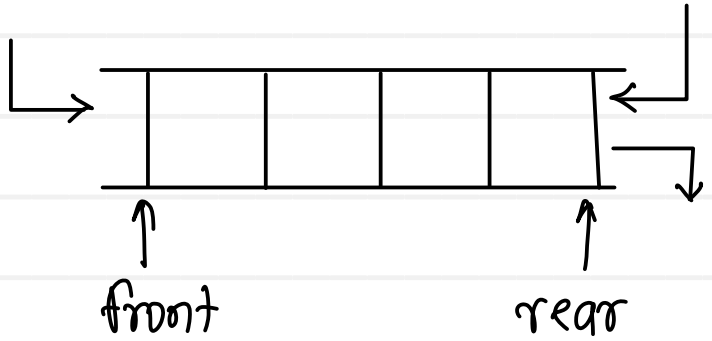


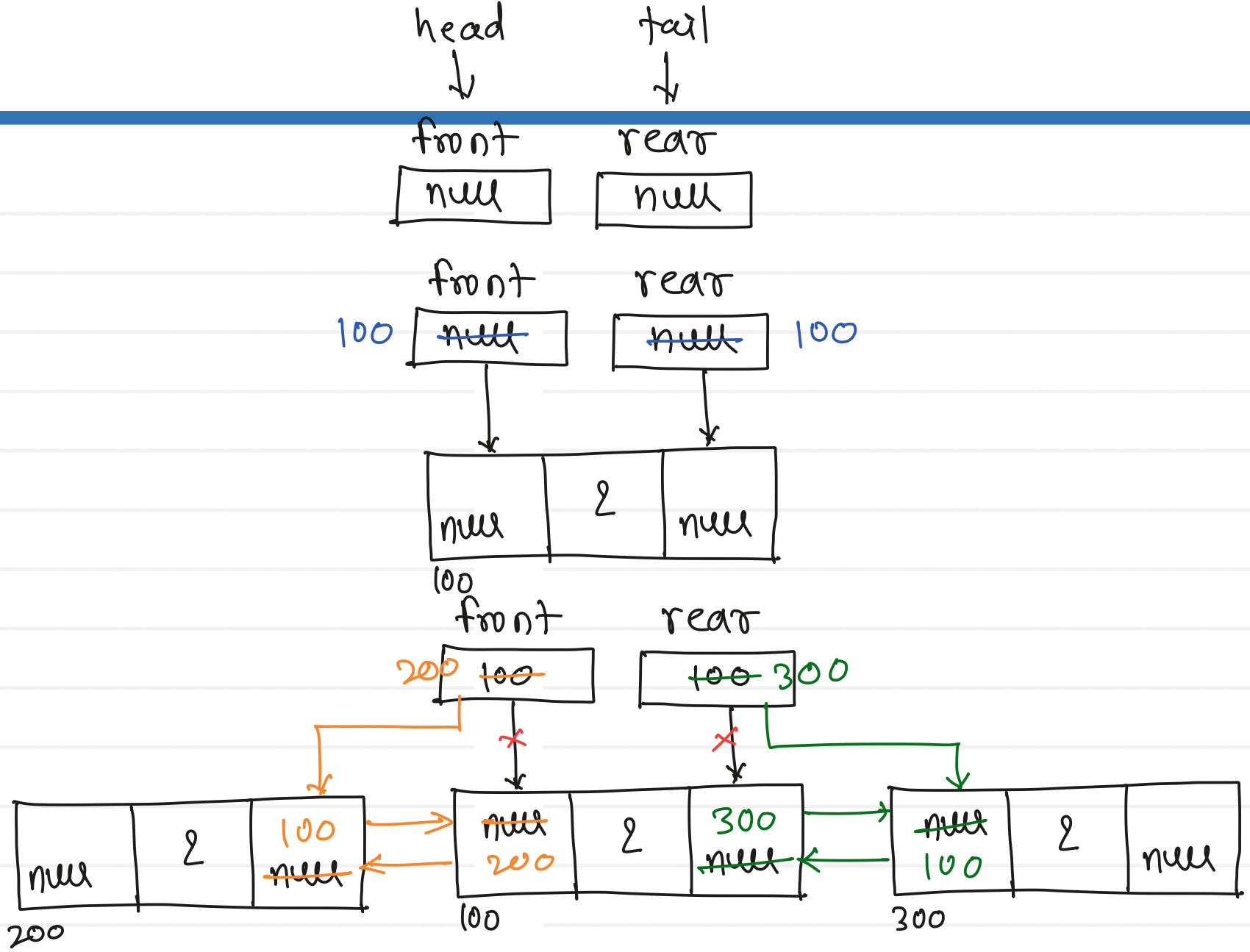
Output Restricted Deque

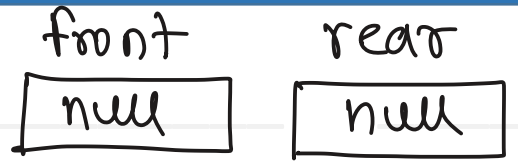
insert from both ends

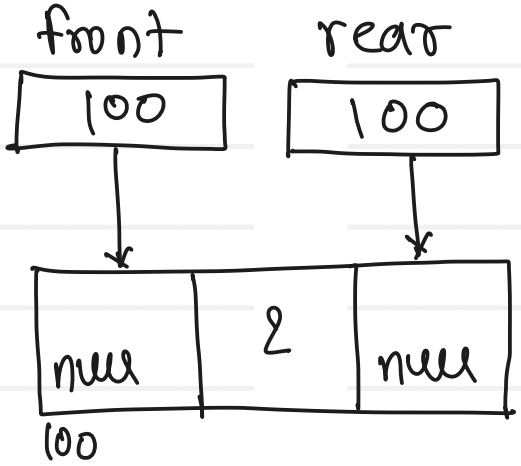
remove from only one end

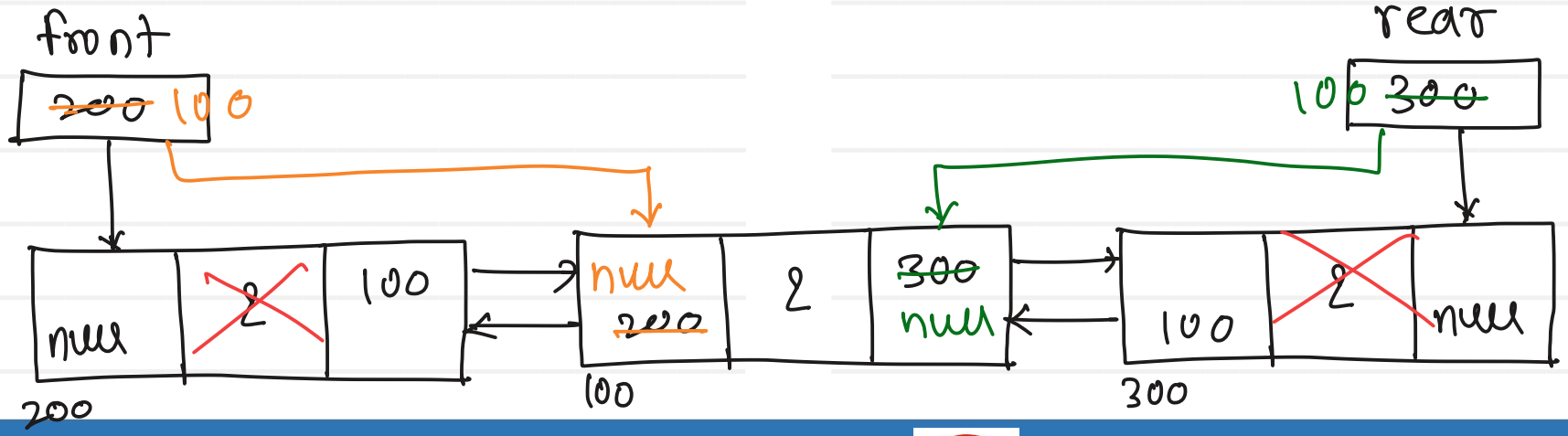





head
tail
front
rear
null
null
front
rear
100
null
null
100
null
2
null
100
front
rear
200
100
100
300
null
2
100
null
200
null
2
300
null
100
null
2
null
200
100
300


front
rear
null
null

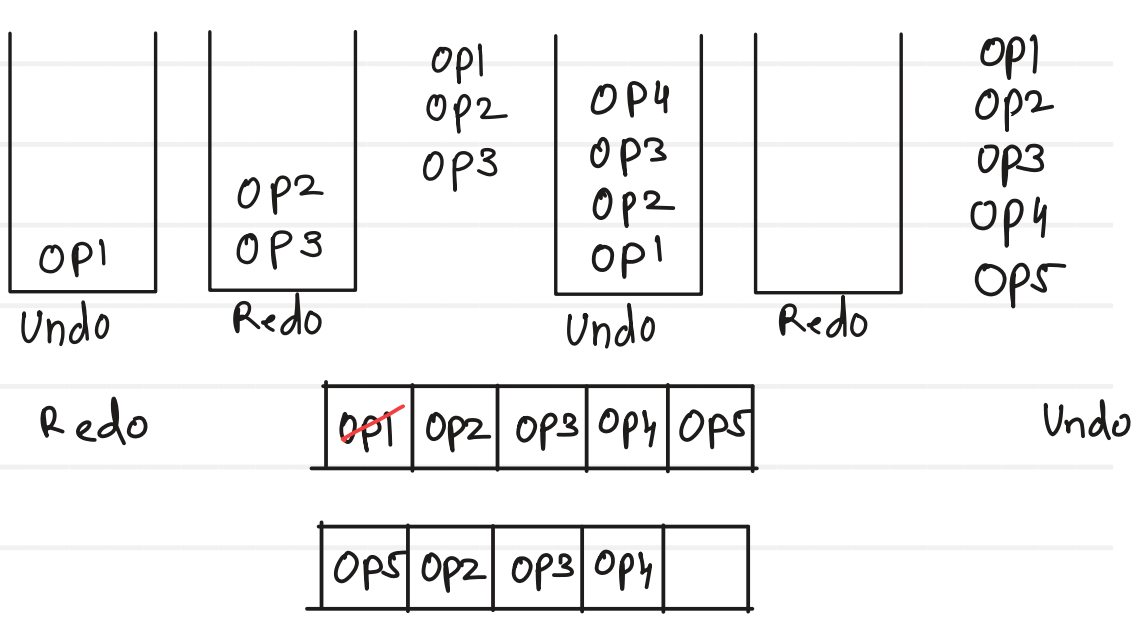

front
rear
100
100
null
2
null
100


front
~~200~~ 100
rear
100 ~~300~~
null
2
100
200
null
~~200~~
2
~~300~~
null
100
100
2
null
300

Applications of Deque

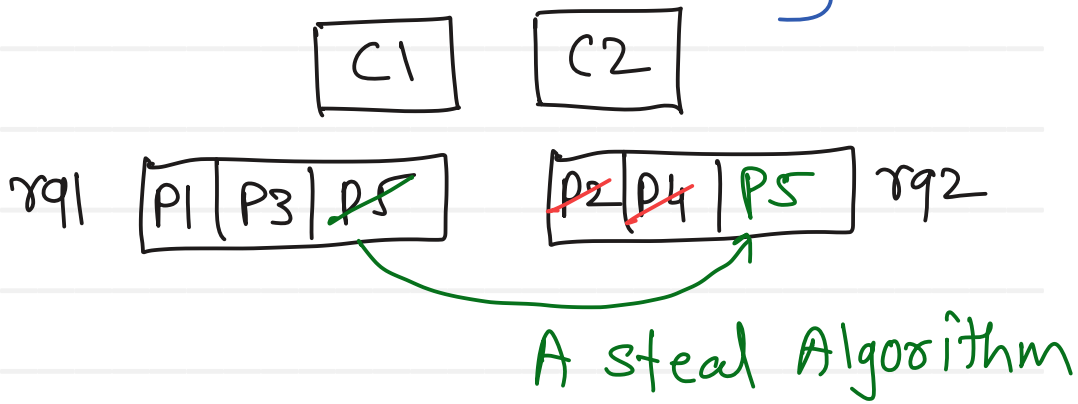
1. stack and queue are efficiently implemented using deque.

2. Undo & redo operations are performed efficiently using deque.



3. Browser history is also managed with deque (forward & Backwork)

4. Scheduling in multiprocessor environment is done using deque

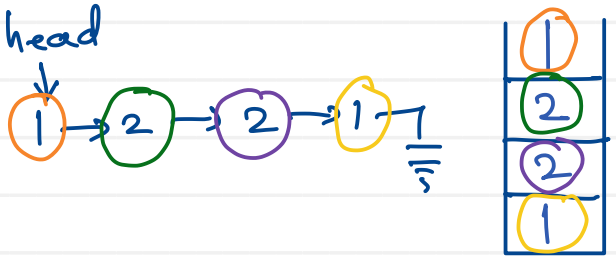


5. It is also used to check palindromes (in case of linked List)

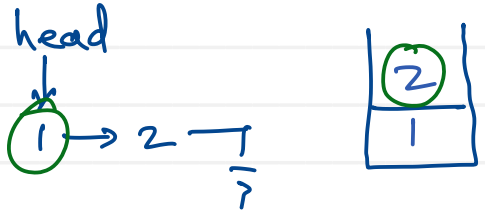
Palindrome Linked List

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:
Input: head = [1,2,2,1]
Output: true



Example 2:
Input: head = [1,2]
Output: false



to push – n
to compare – n
T ∝ 2n

T(n) = O(n)

stack size = n
S ∝ n

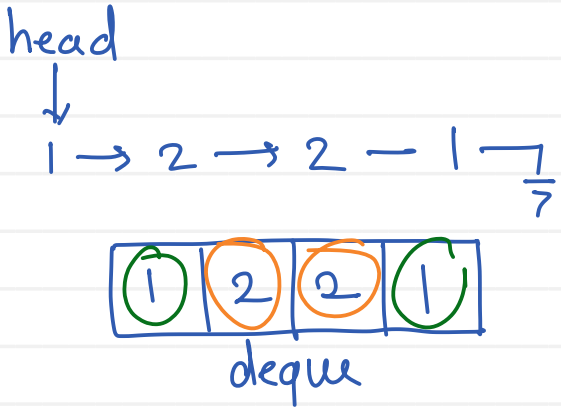
S(n) = O(n)

```
boolean isPalindrome(Node head) {
    Stack<Integer> st = new Stack<>();
    Node trav = head;
    while (trav != null) {
        st.push(trav.data);
        trav = trav.next;
    }
    Node trav = head;
    while(!st.isEmpty()) {
        if(trav.data != st.pop())
            return false;
        trav = trav.next;
    }
    return true;
}
```

Palindrome Linked List

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:
Input: head = [1,2,2,1]
Output: true



Example 2:
Input: head = [1,2]
Output: false

to push – n
to compare – n/2

$T \propto 3/2\ n$

$T(n) = O(n)$

deque size = n

$S \propto n$

$S(n) = O(n)$

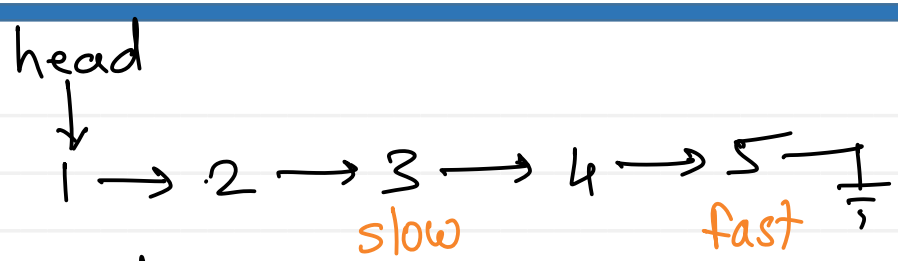
Middle of the Linked List

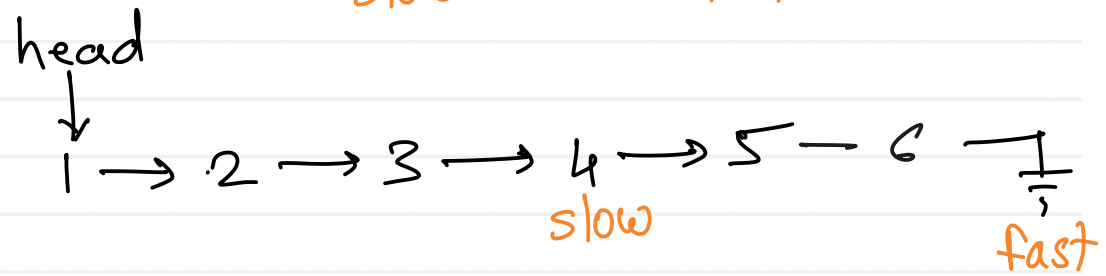
Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.

Example 2:
Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.





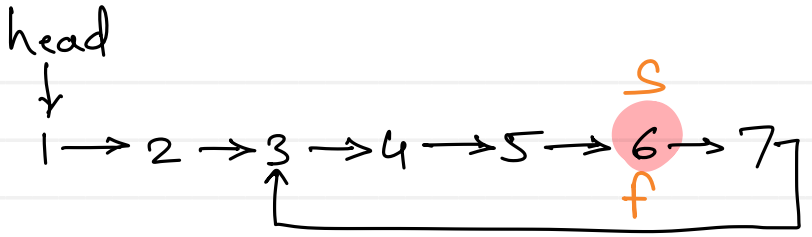

```
Node slow = head;
Node fast = head;
while(fast != null && fast.next != null){
    slow = slow.next;
    fast = fast.next.next;
}
sysout(slow.data)
```

Linked List Cycle

Given head, the head of a linked list, determine if the linked list has a cycle in it.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

Return true if there is a cycle in the linked list. Otherwise, return false.



Example 1:
Input: head = [3,2,0,-4], pos = 1
Output: true

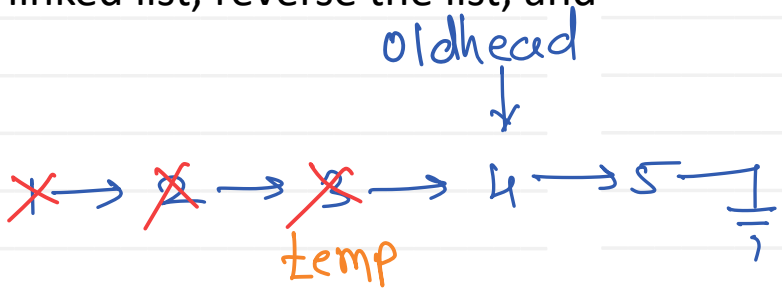
Example 2:
Input: head = [1,2], pos = 0
Output: true

Example 3:
Input: head = [1], pos = -1
Output: false

```
Node slow = head;
Node fast = head;
while(fast != null && fast.next != null){
    slow = slow.next;
    fast = fast.next.next;
    if( fast == slow)
        return true;
}
return false;
```

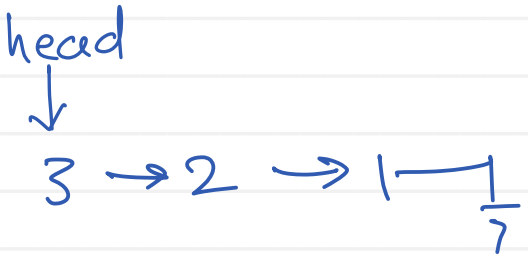
Reverse Linked List

Given the head of a singly linked list, reverse the list, and return the reversed list.



Example 1:
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]

Example 2:
Input: head = [1,2]
Output: [2,1]



Example 3:

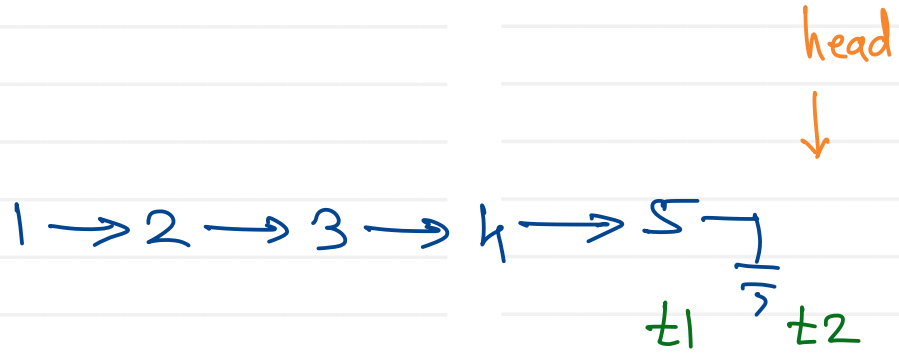
Input: head = []
Output: []

```
Node oldhead = head;
head = null;
while( oldhead != null) {
    Node temp = oldhead;
    oldhead = oldhead.next;
    temp.next = head;
    head = temp;
}
```

Reverse Linked List

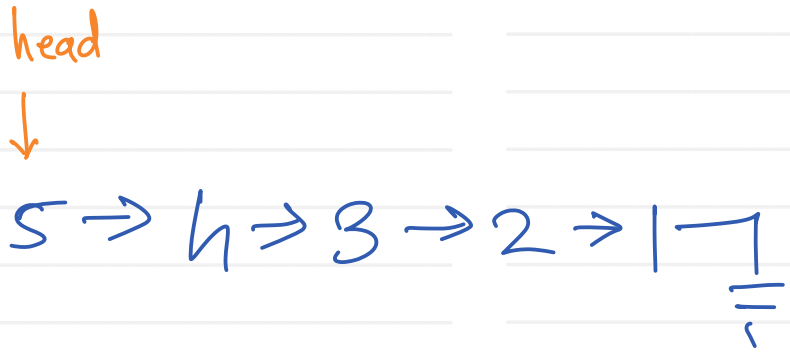
Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]



Example 2:
Input: head = [1,2]
Output: [2,1]
Example 3:

Input: head = []
Output: []

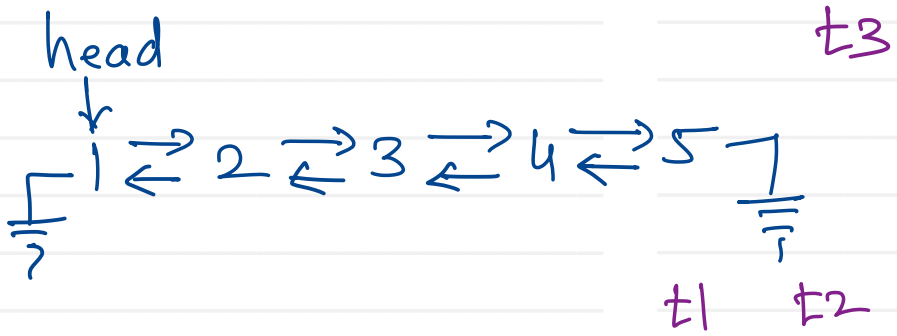



```
t1 = null;
t2 = head;
while (t2 != null) {
    head = t2.next;
    t2.next = t1;
    t1 = t2;
    t2 = head;
}
head = t1;
```

Reverse Linked List

Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]



Example 2:
Input: head = [1,2]
Output: [2,1]

Example 3:



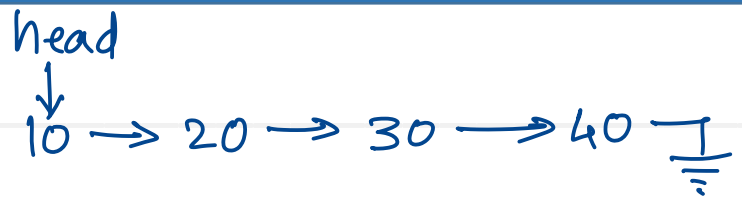
Input: head = []
Output: []

T(n) = O(n)

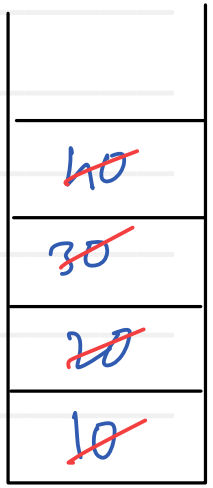
S(n) = O(1)

```
Node reverseList(head){
    Node t1 = head;
    Node t2 = head.next;
    Node t3;
    t1.next = null;
    while(t2 != null){
        t3 = t2.next;
        t2.next = t1;
        t1.prev = t2;
        t1 = t2;
        t2 = t3;
    }
    head = t1;
    return head;
}
```

Singly linear linked list - Display reverse



Output : 40 30 20 10



1. push all node values on stack
2. pop & print all values from stack

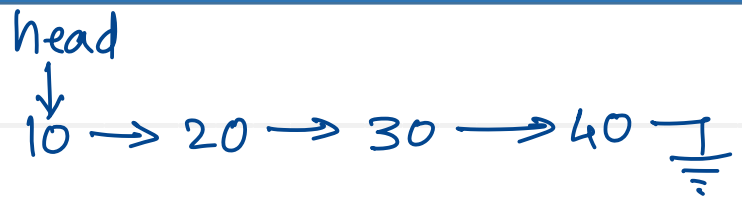
```
void reverseDisplay( ) {
    Stack<Integer> st = new Stack<>( );
    Node trav = head;
    while ( trav != null) {          <- n times
        st.push( trav.data);
        trav = trav.next;
    }
    while( !st.isEmpty( ))           <- n times
        sysout( st.pop( ));
}
```

$T \propto 2n$ time

$S(n) = O(n)$

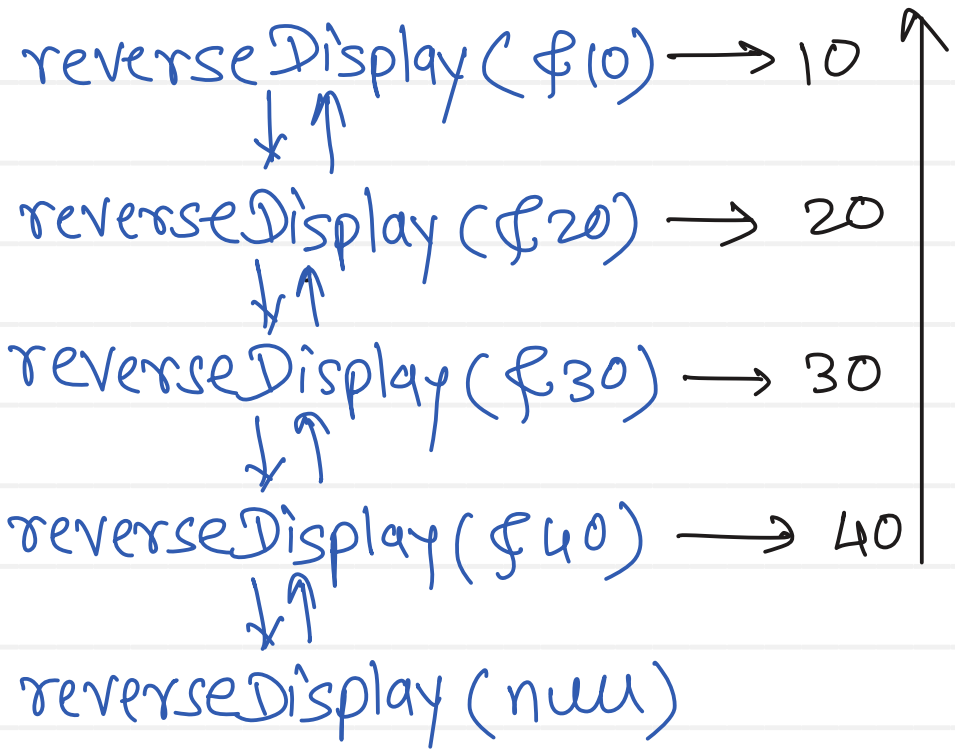
$T(n) = O(n)$

Singly linear linked list - Display reverse



Output : 40 30 20 10

```
void reverseDisplay ( Node trav) {
    if ( trav == null)
        return;
    reverseDisplay ( trav.next);
    sysout ( trav.data);
}
```



$T(n) = O(n)$

$S(n) = O(n)$

```
void reverseDisplay(Node trav) {
    if (trav == null)
        return;
    reverseDisplay(trav.next);
    sysout(trav.data)
}
```

non tail (head) recursion

```
void forwardDisplay(Node trav) {
    if (trav == null)
        return;
    sysout(trav.data)
    forwardDisplay(trav.next);
}
```

tail recursion